

Exercise 4

Merge sort

Now we're going to implement sorting of lists using the [merge sort](#) algorithm. Merge sort is a divide-and-conquer algorithm that is built out of three parts.

1. Dividing a list into two sublists that are to be sorted.
2. Sorting the sublists
3. Merging two sublists that are already sorted

This has a very natural recursive definition and a succinct Haskell implementation.

We'll break this into parts. The template code for this exercise is in [lists-mergesort.hs](#)

First, define a function

```
halve :: [a] -> ([a], [a])
```

which splits a list into two halves at its midpoint (for odd-length lists the two sublists should have lengths that differ by at most one). You might find [splitAt :: Int -> \[a\] -> \(\[a\], \[a\]\)](#) helpful.

Next define a function

```
merge :: Ord a => [a] -> [a] -> [a]
```

which takes two (sorted) lists and merges them into one sorted list. This is probably easiest to write recursively, think about the possible cases.

Finally use your two helper functions to write

```
mergeSort :: Ord a => [a] -> [a]
```

Again, the recursive definition is a natural one. There are base cases for empty and singleton lists, while the recursive case should implement the divide and conquer, splitting and merging sorted sublists.

A higher-order version

Notice how this implementation of `mergeSort` requires that the list entries are [orderable](#). It is often useful to instead allow the caller to provide a comparison function that orders elements.

Rework your code to implement

```
mergeSortWith :: (a -> a -> Ordering) -> [a] -> [a]
```

Where the `Ordering` type has three values `LT`, `EQ`, or `GT`.

You'll need to implement new `mergeWith` and `mergeSortWith` functions. With `mergeSortWith` implemented, the implementation of `mergeSort` is then just

```
mergeSort' :: Ord a => [a] -> [a]
mergeSort' = mergeSortWith compare
```

Where we used the generic function [compare](#).

Question 4.1

Do you understand how the implementation of `mergeSort'` works using currying and partial application?

Composition

We can use this idea of higher-order functions and composition as building blocks for a number of different ways of sorting objects.

To simplify spelling things, let's introduce a type-synonym for the type of the comparison function

```
type Comparator a = a -> a -> Ordering
```

We can now do various things, suppose that we want to sort lists in a reverse order from the one given by the default ordering. We could do

```
sortReverse :: Ord a => [a] -> [a]
sortReverse = reverse . mergeSortWith compare
```

But this unnecessarily traverses the list to reverse it at the end.

Better is to invert the comparison function. I provide an `invertOrdering` function that reverses an ordering.

Exercise 4.2

Implement

```
invert :: Comparator a -> Comparator a
```

Which takes a comparator and produces a new comparator that delivers the reverse. We can then implement `sortReverse` without the extra traversal

```
sortReverse :: Ord a => [a] -> [a]
sortReverse xs = mergeSortWith (invert compare) xs
```

The `invert` function *returns* a function, so we need a way to introduce names for its arguments (they don't appear on the left hand side of the definition).

The easiest way to do this is by using a lambda expression on the right hand side. If you want to then see how write it in [pointfree](http://pointfree.io) style, go to pointfree.io.

Now let's look at slightly more involved comparator transformations.

Suppose we have a `Comparator a` and a way of turning `bs` into `as`, we can use this to deliver a `Comparator b`. We'll call this function `on` for reasons which will become (hopefully) obvious.

```
on :: Comparator a -> (b -> a) -> Comparator b
on = undefined
```

For example, suppose we want to compare tuples by their first element using the builtin `compare` on that element.

Our comparison operation would be

```
on compare fst
```

Now we can see why we called this function `on`, since when we write it with infix notation

```
compare `on` fst
```

We read this as "Compare the elements on their first part". As an example, if we run

```
Prelude> mergeSortWith (compare `on` fst) [(7, "a"), (-1, "b"), (5, "d")]
[(-1, "b"), (5, "d"), (7, "a")]
```

Exercise 4.3

Implement `on`. Check that it produces the right answer.

Like with `invert`, the `on` function *returns* a function, so we need a way to introduce names for its arguments (they don't appear on the left hand side of the definition).

The easiest way to do this is by using a lambda expression on the right hand side. If you want to then see how write it in [pointfree](http://pointfree.io) style, go to pointfree.io.

If the transformation function you provide is the identity function `id`, then you should get back the same thing as if you had not used `on`. This is a useful sanity check.

That is

```
compare `on` id == compare
```

Exercise 4.4

Finally, we'll compose these two extra functions. Write a function

```
sortReversedByLengthSnd :: [(a, String)] -> [(a, String)]
```

That sorts a list of pairs of something and strings in reverse order by the length of the string.

For example

```
Prelude> sortReversedByLengthSnd [(5, "a"), (2, "foo"), (3, "four"),  
(7, "ab")]  
[(3, "four"), (2, "foo"), (7, "ab"), (5, "a")]
```

You should not write this comparison function by hand, but just compose together functions which you already have.